

GannaApp

Sugarcane Mill Indent Planning System

COMPREHENSIVE DOCUMENTATION

Everything You Need to Know — Business Context, Algorithm, Application, and Testing

Generated: 05 June 2026

Table of Contents

Part 1 The Business — Sugarcane Milling and Indent Planning

- 1.1 What is a Sugarcane Mill?
- 1.2 What is an Indent?
- 1.3 Why is Planning Difficult?
- 1.4 The 3-Day Lead Time Problem (T+3)
- 1.5 Key Business Vocabulary

Part 2 The Data — Where It Comes From and What It Means

- 2.1 Bonding Data
- 2.2 Indent Data
- 2.3 Purchase Data
- 2.4 Center Mapping
- 2.5 Yard Balance Files
- 2.6 Standard Yard Balance

Part 3 The Algorithm — How the Calculation Works

- 3.1 Overview: What the Algorithm Produces
- 3.2 Step 1 — Maturity Rates (D-Weights)
- 3.3 Step 2 — Adjusted Requirement per Center
- 3.4 Step 3 — Forecasting Arrivals from Open Indents
- 3.5 Step 4 — Computing the Final Indent
- 3.6 The Overrun Factor
- 3.7 The Complete Formula

Part 4 The Application Architecture

- 4.1 Technology Stack
- 4.2 Frontend Overview
- 4.3 Backend Overview
- 4.4 Database

Part 5 User Interface — Step-by-Step Walk-through

- 5.1 Logging In
- 5.2 Dashboard (Mission Control)
- 5.3 Step 1 — Uploading Data
- 5.4 Step 2 — Setting Parameters
- 5.5 Step 3 — Review and Calculate
- 5.6 Step 4 — Reading the Results
- 5.7 History Page

Part 6 Technical Deep Dive — Code Walkthrough

- 6.1 calculationService.ts — The Brain
- 6.2 Data Normalization
- 6.3 D-Weight Computation in Code
- 6.4 Forecast Computation in Code
- 6.5 Final Indent Computation in Code

Part 7 Testing and Validation

7.1 The Excel Reference Model

7.2 Test Case — December 12, 2025 (GATE)

7.3 How to Run the Test

Part 8 Roles, Permissions, and Multi-Tenancy

Part 9 Frequently Asked Questions

PART 1 — The Business: Sugarcane Milling and Indent Planning

1.1 What is a Sugarcane Mill?

A sugarcane mill is a large industrial facility that processes raw sugarcane into sugar, molasses, and other by-products. The mill operates continuously during the "crushing season" — typically from October to March in India. During this period, the mill must receive a steady, daily supply of raw sugarcane to keep its machines running at full capacity.

Sugarcane is grown by thousands of farmers spread across dozens of "centers" (collection points) in the surrounding region. Each center is a geographic location — often a village or cluster of villages — where farmers bring their harvested cane. The mill dispatches trucks or trolleys to these centers to pick up the cane and bring it to the mill.

The mill has a daily "crushing capacity" — the maximum amount of sugarcane (measured in Quintals, abbreviated Qtl) it can process in 24 hours. Falling below capacity means the mill earns less revenue. Receiving too much cane causes yard overflow, spoilage (sugarcane loses sugar content quickly after cutting), and logistical chaos. The goal is to receive exactly the right amount of cane every day.

 *UNIT: Quintal (Qtl) = 100 kilograms. A typical sugarcane mill might process 5,000 to 15,000 Quintals per day. The daily crushing target in this application is typically around 80,000 Quintals.*

1.2 What is an Indent?

An "indent" is a formal purchase order or instruction sent by the mill to a specific center, telling it: "Please prepare and deliver X Quintals of sugarcane on a specific date." The word "indent" comes from the old practice of indenting (stamping) official purchase requisitions.

Each indent specifies:

- Which center it is for (e.g., "GATE" center, "BILAULI" center)
- The date for which the cane is "raised" (the indent date)
- The quantity in Quintals that should arrive

When the mill raises an indent for a center, farmers at that center cut their cane, load it, and begin transporting it to the mill. The cane does not all arrive on the same day — it dribbles in over 3 to 5 days depending on the distance of the farmer, road conditions, and the number of vehicles available. This staggered arrival pattern is called the "maturity pattern" of the indent.

1.3 Why is Planning Difficult?

Indent planning is surprisingly complex for several reasons:

1. Delayed Arrivals — The Lead Time Problem

When the mill raises an indent today, the bulk of the cane will not arrive today. It arrives over the next 3–5 days. The mill must therefore decide TODAY how much cane to order so that exactly the right amount

arrives 3 days from now (T+3). If the mill orders too little, the mill may sit idle in 3 days. If it orders too much, cane piles up in the yard, spoils, and causes problems.

2. Multiple Centers — Distributed Supply

A typical mill area might have 50 to 150+ centers. Each center has a different bonding (annual commitment), different distance from the mill, different road conditions, and different maturity patterns. The mill must plan indents for ALL centers simultaneously, allocating the total daily requirement proportionally based on each center's bonding.

3. Variable Maturity — The D-Weight Problem

Even for the same center, the fraction of cane that arrives on Day 1 vs Day 2 vs Day 3+ of an indent varies from one indent to the next. Weather, vehicle availability, and harvest timing all affect this. The planning algorithm must estimate this maturity pattern from historical data.

4. Existing Open Indents — Multiple Indents in Transit

At any given moment, there are typically 3–5 "open" indents for each center — indents that were raised on previous days and whose cane is still arriving. Before deciding how much to indent today, the planner must forecast how much cane will arrive from those open indents on T+3. Only the SHORTFALL needs to be covered by today's new indent.

5. Yard Balance — Stock Adjustment

The mill maintains a physical "yard" where cane is stored before crushing. If the yard has more cane than the target stock level, the mill can indent less. If the yard is below target, it must indent more. This daily stock adjustment affects every center's indent proportionally.

1.4 The 3-Day Lead Time Problem (T+3)

The central concept in this application is the "T+3" framework. Here, T represents the current date (the day the planner is running the calculation). The notation works as follows:

Notation	Meaning	Example (if T = Dec 12)
T	Today — the date the calculation is run	December 12, 2025
T+1	Tomorrow	December 13, 2025
T+2	Day after tomorrow	December 14, 2025
T+3	Three days from today — the TARGET delivery date	December 15, 2025
T-1	Yesterday	December 11, 2025
T-4	Four days ago — oldest "recent closed" indent	December 8, 2025
T-7	Seven days ago — boundary of recent history	December 5, 2025

The indent raised TODAY (T) is "for" T+3. That means the mill wants cane from today's indent to START arriving on December 15. However, the cane from today's indent does not arrive all at once — it arrives over Days 1 through 4+ relative to the indent date:

Day Bucket	Arrival Day Relative to Indent Date	Example for T=Dec 12 Indent
D1	Purchase Date ≤ Indent Date (day of or	Arrives Dec 15 (or earlier)

	earlier)	
D2	Purchase Date = Indent Date + 1 day	Arrives Dec 16
D3	Purchase Date = Indent Date + 2 days	Arrives Dec 17
D4	Purchase Date = Indent Date + 3 or more days	Arrives Dec 18 onwards

Notice that D1 includes all cane that arrives ON OR BEFORE the indent date — this is because in practice, some cane from a given indent starts moving even before the official indent date (farmers start cutting early). The D1 bucket is therefore the "first wave" of arrivals.

1.5 Key Business Vocabulary

Term	Plain English Meaning
Indent	A purchase order telling a center how much cane to prepare for delivery
Purchy (Purchase)	An actual delivery of cane recorded when it arrives at the mill gate
Center	A geographic collection point where farmers bring their harvested cane
Bonding	The seasonal commitment — how many total Quintals of cane a center has agreed to supply for the entire season
Quintal (Qtl)	100 kilograms — the unit of measurement for all cane quantities
Maturity Pattern	The fraction of an indent that arrives on each day (D1, D2, D3, D4)
D-Weights (D1,D2,D3,D4)	The maturity fractions as decimals. E.g., D1=0.45 means 45% of indented quantity arrives on or before the indent date
Overrun	When the total cane actually received exceeds the indented quantity. $\text{Overrun} = (\text{total purchases} / \text{total indents}) - 1$
Yard Balance	The amount of cane physically sitting in the mill yard at any given time
T	The current date — the date on which the calculation is performed
T+3	The target delivery date — three days from today
Open Indent	An indent whose cane is still arriving (has not fully closed yet)
Closed Indent	An indent whose delivery window has fully passed (used for historical analysis)
Gate Center	The primary, central collection point closest to the mill (code = 1, named GATE)
Season	The crushing season, typically October to March each year
Crushing Capacity	The maximum Quintals the mill can process per day

PART 2 — The Data: Where It Comes From and What It Means

The application works with six types of data files. Each is a CSV (Comma-Separated Values) file that the user uploads once per season (or daily for the yard balance). Understanding what each file contains is essential for understanding the calculation.

2.1 Bonding Data

The bonding file defines the SEASONAL COMMITMENT of every center. At the start of each crushing season, each farmer signs a contract with the mill committing to supply a certain number of Quintals of sugarcane over the entire season. The bonding file aggregates these individual farmer commitments into per-center totals.

File format (CSV columns):

Column	Example	Meaning
Code	1	Unique numeric ID of the center
Center Name	GATE	Human-readable name of the center
Bonding	4,126,403.82	Total Quintals committed by all farmers in this center for the entire season

💡 **WHY BONDING MATTERS:** The bonding figure determines each center's *SHARE* of the total daily requirement. A center with bonding = 4,126,403 in a total bonded pool of 17,266,808 Qtl has a share of 23.9% — meaning it should receive roughly 23.9% of the daily indent target. Bonding is the most fundamental input to the calculation.

The GATE center (code 1) typically has the largest bonding because it represents the primary supply zone closest to the mill. In the current season data, GATE's bonding is approximately 4.1 million Quintals out of a total ~17.3 million Quintals — making GATE responsible for about 24% of all supply.

2.2 Indent Data

The indent file is the historical record of every purchase order the mill has raised during the current season. It grows daily as new indents are raised.

File format (CSV columns):

Column	Example	Meaning
Code	1	Center ID
Center Name	GATE	Center name
Indent Date	24-10-2025	The date for which this indent was raised (DD-MM-YYYY)
No of Purchy	3	Number of individual purchase transactions
Qty in Qtls	99	Total Quintals indented for this center on this date

Each row represents ONE center's indent for ONE date. If the mill operates 100 centers and has been running for 50 days, this file would have approximately 5,000 rows. The application normalizes this

data so that if the same center appears twice for the same date (data entry error or split batch), the quantities are summed into a single record.

IMPORTANT: The indent date is the "raised for" date — the date on which the cane from this indent should START arriving (the D1 date). It is NOT the date the indent was placed. There is always a planning offset: the planner places the indent today for the cane to arrive 3 days later. So an indent placed on Dec 12 has an indent date of Dec 15.

2.3 Purchase Data

The purchase file records every actual delivery of cane to the mill. Every time a vehicle arrives with cane from a center, the mill gate records a "purchase" entry.

File format (CSV columns):

Column	Example	Meaning
Code	1	Center ID of the arriving cane
Center Name	GATE	Center name
Purchase Date	24-10-2025	The actual date this cane arrived at the mill
Indent Date	24-10-2025	The indent date this purchase belongs to (links to indent file)
No of Purchy	2	Number of vehicle trips in this batch
Qty in Qtls	80.59	Quintals received in this batch

The linkage between purchase date and indent date is critical. For example, an indent raised for center GATE on December 5 will have purchases trickling in on December 5 (D1), December 6 (D2), December 7 (D3), and December 8+ (D4). By comparing the purchase date to the indent date, the algorithm can compute the exact "day fraction" for each arrival.

 **KEY RELATIONSHIP:** The "Indent Date" column in the purchase file links each purchase back to its originating indent. This is how the algorithm knows "this cane arriving today on December 8 was originally ordered (indented) on December 5."

2.4 Center Mapping

The center mapping file handles the situation where some smaller or satellite centers are physically consolidated into a larger, nearby center. This happens when the mill decides to route all cane from a small center through a larger collection point.

File format (CSV columns):

Column	Example	Meaning
Source	28	The center whose data should be merged
Target	23	The center it should be merged INTO

Example mappings in the current season: 28→23, 75→112, 144→136, 178→496.

When center 28 maps to center 23, this means: ALL indent records for center 28 are summed into center 23's indent. ALL purchase records for center 28 are counted as center 23's purchases. Center

28 disappears from the output — it is physically merged into center 23. The bonding for center 28 is also added to center 23's bonding total.

 **CENTER MAPPING = PHYSICAL AGGREGATION:** This is not just a label change. All data (indent qty, purchase qty, bonding) from the source center is **ADDED** to the target center. The source center no longer appears in results.

2.5 Yard Balance (Daily Actual)

The yard balance file records the actual physical stock of cane sitting in the mill yard at the end of each day. It is updated daily and serves as the "available stock" input to the calculation.

File format (CSV columns):

Column	Example	Meaning
Date	12-12-2025	The date of this yard balance reading
Gate	1767	Quintals of cane in the yard from Gate-type centers
Centre	17161	Quintals of cane in the yard from non-Gate centers

The application automatically finds the most recent entry in this file with a date on or before the current calculation date, and uses those Gate and Centre values as the "available stock" inputs for the stock adjustment calculation.

For example, if the current date is December 12, 2025, and the file has entries through December 11, the application will use December 11's Gate=1767 and Centre=17161 as the available stock figures. These are then compared to the "standard" (target) yard balance to compute whether the mill needs more or less cane than the bonding-based requirement alone.

2.6 Standard Yard Balance (Target Stock)

The standard yard balance file defines the TARGET stock level the mill wants to maintain in the yard at all times. This is a fixed, rarely-changing reference file.

Type	Standard Quantity	Meaning
Gate	9,000 Qtl	The mill wants to always have 9,000 Qtl from Gate-area centers in the yard
Centre	6,000 Qtl	The mill wants to always have 6,000 Qtl from other centers in the yard

If the actual yard balance (from the daily file) is **BELOW** the standard, the mill needs to increase today's indent to replenish the yard. If it is **ABOVE** the standard (surplus), the mill can reduce today's indent slightly. This is the "stock adjustment" component of the calculation.

PART 3 — The Algorithm: How the Calculation Works

3.1 Overview: What the Algorithm Produces

The algorithm takes all the uploaded data and parameters as input, and produces for each center:

- The recommended indent quantity in Quintals to raise today (for delivery on T+3)
- A detailed breakdown of every calculation step
- A forecast matrix showing expected cane arrivals over the next several days
- A maturity analysis showing the historical D-weight rates per center

The algorithm runs entirely in the browser (frontend JavaScript). No server computation is required for the calculation itself — the server only stores the input data and saves the completed results.

3.2 Step 1 — Maturity Rates (D-Weights)

Before the algorithm can recommend any indent, it must first understand the "maturity pattern" of each center — what fraction of an indent arrives on Day 1, Day 2, Day 3, and Day 4+. These fractions are called D-weights.

What are D-Weights?

For a given historical indent, the D-weights are computed by dividing the actual purchases by the indent quantity:

D-Weight	Formula	Interpretation
D1	$D1_purchases \div indent_qty$	Fraction arriving on or before the indent date
D2	$D2_purchases \div indent_qty$	Fraction arriving exactly 1 day after indent date
D3	$D3_purchases \div indent_qty$	Fraction arriving exactly 2 days after indent date
D4	$D4_purchases \div indent_qty$	Fraction arriving 3 or more days after indent date

 **EXAMPLE:** Center GATE raised an indent of 21,411 Qtl for December 5. Actual purchases were: Dec 5 = 9,129 Qtl (D1), Dec 6 = 7,334 Qtl (D2), Dec 7 = 4,587 Qtl (D3), Dec 8+ = 1,344 Qtl (D4). Then:

$D1 = 9,129 \div 21,411 = 0.4263$ (42.63% arrived on Day 1)

$D2 = 7,334 \div 21,411 = 0.3426$ (34.26% arrived on Day 2)

$D3 = 4,587 \div 21,411 = 0.2142$ (21.42% arrived on Day 3)

$D4 = 1,344 \div 21,411 = 0.0628$ (6.28% arrived on Day 4+)

Which Indents Are Used?

The algorithm uses the most recent CLOSED indents for each center. A closed indent is one whose entire delivery window has passed. Specifically, the algorithm looks at indents with indent dates between T-7 (seven days ago) and T-4 (four days ago). This gives up to 4 recent data points per center:

Indent Date	Days Before T	Status	Use in D-Weight Calculation
-------------	---------------	--------	-----------------------------

T-7	7 days ago	Fully closed — all D4 cane received	YES — used
T-6	6 days ago	Fully closed	YES — used
T-5	5 days ago	Fully closed	YES — used
T-4	4 days ago	Effectively closed (D4 started T-1)	YES — used
T-3	3 days ago	D4 started today	NO — still partially open
T-2	2 days ago	D3 received, D4 not started	NO — open indent
T-1	1 day ago	D2 received, D3/D4 pending	NO — open indent
T (Today)	0 days	D1 arriving	NO — newly raised

How D-Weights Are Averaged:

For each center, the D-weights from its recent closed indents (up to 4) are averaged using a SIMPLE average (arithmetic mean). This means each indent has equal weight regardless of its size.

For example, if GATE has 4 recent closed indents with D1 rates of 0.4263, 0.4702, 0.4342, and 0.4534, the average D1 weight is:

$$avg_D1 = (0.4263 + 0.4702 + 0.4342 + 0.4534) \div 4 = 0.4460$$

This is a SIMPLE average, not a weighted average. The Excel model uses simple average, and this application matches that exactly.

D-Weight Fallback:

If a center has no recent closed indent data (because it is a small center that did not indent every day, or the season just started), the algorithm falls back to using the SEASON-WIDE average D-weights for that center — computed from all indents since the plant start date. If even season data is missing, the center receives a 0 indent recommendation.

3.3 Step 2 — Adjusted Requirement per Center

Once we have D-weights, the algorithm computes how much cane SHOULD arrive at the mill on T+3 from each center. This "adjusted requirement" has three components:

Component A: Requirement by Bonding

The mill has a total daily requirement (e.g., 80,000 Qtl per day at 80% plant capacity). This requirement is split among all centers in proportion to their bonding:

$$effective_requirement = daily_requirement \times (plant_capacity \div 100)$$

$$requirement_by_bonding = effective_requirement \times (center_bonding \div total_all_bonding)$$

For GATE with bonding = 4,126,403 Qtl and total bonding = 17,266,808 Qtl:

$$bonding_share = 4,126,403 \div 17,266,808 = 23.9\%$$

$$requirement_by_bonding = 80,000 \times 23.9\% = 19,118 \text{ Qtl}$$

Component B: Stock Adjustment

The mill's yard balance affects the indent. If the yard has LESS cane than the target (standard) level, the mill needs to order more to replenish. If it has MORE, it can order less. The stock adjustment distributes the yard surplus or deficit across centers proportionally to their bonding within their type (Gate or non-Gate):

$$\text{stock_diff_gate} = \text{standard_stock_gate} - \text{available_stock_gate}$$

$$\text{stock_diff_centre} = \text{standard_stock_centre} - \text{available_stock_centre}$$

For GATE centers:

$$\text{stock_adjustment} = \text{stock_diff_gate} \times (\text{center_bonding} \div \text{total_gate_bonding})$$

For non-GATE centers:

$$\text{stock_adjustment} = \text{stock_diff_centre} \times (\text{center_bonding} \div \text{total_centre_bonding})$$

Since GATE is the only "gate" center, total_gate_bonding = GATE's bonding, and stock_adjustment for GATE = stock_diff_gate × 1.0 = the full gate stock deficit/surplus.

 **EXAMPLE** (Dec 12, 2025): Standard stock gate = 9,000 Qtl. Available stock gate = 1,767 Qtl (from the yard balance file for Dec 11). Stock diff = 9,000 – 1,767 = +7,233 Qtl. Since total gate bonding = GATE bonding, the stock adjustment for GATE = +7,233 Qtl. This means GATE's requirement increases by 7,233 to replenish the yard.

Component C: Final Adjusted Requirement

$$\text{adjusted_requirement} = \text{requirement_by_bonding} + \text{stock_adjustment}$$

3.4 Step 3 — Forecasting Arrivals from Open Indents

Even before today's new indent is raised, there are already 3 "open" indents for each center — the indents raised on T-2 (for T+1 delivery), T-1 (for T+2 delivery), and T (for T+3 delivery, which is TODAY's new indent, not yet raised). Of these:

- The T+0 indent (indent_date = today, for T+3): the D4 portion will arrive on T+3
- The T+1 indent (indent_date = tomorrow, for T+4 target): the D3 portion arrives on T+3
- The T+2 indent (indent_date = in 2 days, for T+5 target): the D2 portion arrives on T+3

These are called "open indents" — already raised, cane already being cut and loaded, but not yet fully arrived. Some of that already-committed cane will arrive on T+3, reducing how much today's NEW indent needs to cover.

The forecast formula for what will arrive on T+3 from existing open indents:

$$\text{forecastT3} = \text{indent}(T+2) \times D2_avg + \text{indent}(T+1) \times D3_avg + \text{indent}(T+0) \times D4_avg$$

EXAMPLE (Dec 12 for GATE):

$T+0 = \text{Dec 12 indent} = 23,022 \text{ Qtl}; D4_{\text{avg}} = 0.0436 \rightarrow \text{contribution} = 1,003 \text{ Qtl}$

$T+1 = \text{Dec 13 indent} = 27,990 \text{ Qtl}; D3_{\text{avg}} = 0.2123 \rightarrow \text{contribution} = 5,941 \text{ Qtl}$

$T+2 = \text{Dec 14 indent} = 29,610 \text{ Qtl}; D2_{\text{avg}} = 0.3877 \rightarrow \text{contribution} = 11,480 \text{ Qtl}$

$\text{forecastT3} = 1,003 + 5,941 + 11,480 = 18,424 \text{ Qtl already coming to GATE on Dec 15}$

3.5 Step 4 — Computing the Final Indent

Now the algorithm computes the final indent quantity in four sub-steps:

Sub-step A: The Gap

$$\text{gap} = \max(0, \text{adjusted_requirement} - \text{forecastT3})$$

This is how much additional cane is needed on T+3 BEYOND what is already coming from open indents. The $\max(0, \dots)$ ensures a negative gap (meaning more than enough is already coming) results in 0 — we cannot "un-indent".

Example: $\text{gap} = \max(0, 23,862 - 18,424) = 5,438 \text{ Qtl}$

Sub-step B: Target Arrival (Overrun-Adjusted Gap)

$$\text{target_arrival} = \text{gap} \div (1 + \text{overrun_percentage})$$

The "overrun" accounts for the fact that, historically, the total cane received across the season exceeds the total indented quantity. $\text{Overrun} = (\text{total purchases} \div \text{total indents}) - 1$. Since the gap is expressed in terms of "arrivals needed" but D-weights are expressed as ratios to indent quantity, we must convert: divide the gap by $(1 + \text{overrun})$ to get the indent-equivalent target.

Example: $\text{overrun} = 12.44\%$, $\text{target_arrival} = 5,438 \div 1.1244 = 4,837 \text{ Qtl}$

Sub-step C: Final Indent Quantity

$$\text{final_indent} = \text{target_arrival} \div D1_{\text{avg_weight}}$$

Since the D1 portion of the new indent arrives on T+3 (the D1 window covers arrivals on or before the indent date, which is T+3), we need the indent to be large enough that its D1 fraction equals the target arrival.

Example: $\text{final_indent} = 4,837 \div 0.4460 = 10,843 \text{ Qtl}$

This means: if the mill raises an indent of 10,843 Qtl for GATE today, approximately 4,837 Qtl of that will arrive on December 15 (D1), which — combined with the 18,424 Qtl already coming from open indents — gives the adjusted requirement of ~23,261 Qtl.

3.6 The Overrun Factor

Overrun deserves special attention because it is often misunderstood. Overrun occurs when the total cane arriving across the season exceeds what was formally indented. This happens because:

- Farmers sometimes deliver more than their committed quantity
- Some deliveries arrive from indents of OTHER centers (cross-delivery)
- Measurement errors and rounding at the gate scale

- Informal deliveries not linked to any specific indent

The application computes overrun as a SEASON-WIDE figure using all data uploaded:

$$\text{overrun} = (\text{sum of all purchases in season} \div \text{sum of all indents in season}) - 1$$

For the current season (through December 2025), this is approximately 12.44%, meaning for every 100 Qtl indented, the mill receives about 112.44 Qtl. This matches the approach used in the Excel reference model.

3.7 The Complete Formula — Summary

The full calculation for each center can be summarized as:

STEP 1: Compute average D-weights from last 4 closed indents per center

$$D1_avg, D2_avg, D3_avg, D4_avg = \text{simple average of } (dX_purchases / indent_qty)$$

STEP 2: Compute adjusted requirement

$$\text{effective_req} = \text{daily_target} \times \text{plant_capacity}$$

$$\text{req_by_bonding} = \text{effective_req} \times (\text{center_bonding} / \text{total_bonding})$$

$$\text{stock_adjustment} = (\text{standard_stock} - \text{available_stock}) \times (\text{center_bonding} / \text{type_bonding})$$

$$\text{adjusted_req} = \text{req_by_bonding} + \text{stock_adjustment}$$

STEP 3: Forecast arrivals on T+3 from open indents

$$\text{forecastT3} = \text{indent}(T+2) \times D2 + \text{indent}(T+1) \times D3 + \text{indent}(T+0) \times D4$$

STEP 4: Compute final indent

$$\text{gap} = \max(0, \text{adjusted_req} - \text{forecastT3})$$

$$\text{target_arrival} = \text{gap} / (1 + \text{overrun})$$

$$\text{final_indent} = \text{target_arrival} / D1_avg$$

PART 4 — The Application Architecture

4.1 Technology Stack

Layer	Technology	Version	Purpose
Frontend	React	19	User interface components and state management
Frontend	TypeScript	5.x	Type-safe JavaScript for fewer bugs
Frontend	Vite	6.x	Fast development server and build tool
Frontend	Tailwind CSS	3.x	Utility-first CSS for rapid UI styling
Frontend	Lucide React	Latest	Icons used throughout the UI
Backend	Node.js	18+	JavaScript runtime for the server
Backend	Express 5	5.x	Web server framework for API routes
Backend	TypeScript	5.x	Type-safe server code
Database	MySQL	8.x	Relational database for all persistent data
Auth	JWT (JSON Web Tokens)	-	Stateless authentication tokens

4.2 Frontend Overview

The frontend is a single-page application (SPA). All UI rendering happens in the browser. The user sees a seamless, multi-step interface without page reloads.

Key frontend files and their roles:

File/Folder	Role
frontend/App.tsx	Root component — holds ALL application state (data, settings, history). Acts as the "brain" connecting all pages. Routes between Dashboard, Calculator, History, etc.
frontend/types.ts	All TypeScript type definitions — describes every data structure used in the app (Bonding, Indent, Purchase, YardBalanceRow, CalculationInputs, CalculationResults, etc.)
frontend/services/calculationService.ts	The CORE ALGORITHM — all indent calculation logic lives here. Takes uploaded data and parameters, returns recommended indents and detailed breakdowns.
frontend/services/dateUtils.ts	Date parsing and manipulation utilities. Handles multiple date formats (DD-MM-YYYY, YYYY-MM-DD, DD/MM/YYYY).
frontend/services/orgDataService.ts	API client for saving and loading org data files (bonding, indent, purchase, yard balance) from the backend.
frontend/pages/DashboardPage.tsx	The "Mission Control" page. Shows data upload status, recent calculations, and quick actions.
frontend/pages/CalculatorPage.tsx	The 4-step calculation wizard. Orchestrates: upload → parameters → review → results.
frontend/components/Step2Parameters.tsx	The parameter-setting UI — date, plant capacity, stock levels, constraints.
frontend/components/DataManagerModal.tsx	Modal for uploading/replacing/deleting: bonding, indent, purchase, yard balance, and center mapping files.
frontend/components/Step4Results.tsx	The results display page showing the final indent table, forecasts, maturity analysis, and history.

4.3 Backend Overview

The backend is a REST API server built with Express.js. It handles authentication, persists uploaded data per organization, and stores calculation history.

Route	Method	Purpose
/api/auth/login	POST	Verify email+password, return JWT token
/api/auth/signup	POST	Create new user account
/api/org-data/my-org	GET	Load all saved data files for the user's organization
/api/org-data/:type	POST	Save a data file (type = BONDING, INDENT, PURCHASE, YARD_BALANCE)
/api/calculations	GET	Load calculation history for the organization
/api/calculations	POST	Save a completed calculation run
/api/calculations/:id	DELETE	Delete a specific calculation from history
/api/users	GET/POST	User management (admin only)
/api/orgs	GET/POST	Organization management (superadmin only)

4.4 Database

The MySQL database has the following key tables:

Table	What It Stores
organizations	Each client organization: id, name, status (active/suspended), logo
users	User accounts: email, hashed password, name, role, organization_id
org_data_files	The uploaded CSV data per organization. One row per file type per org. Stores the parsed JSON. Types: BONDING, INDENT, PURCHASE, YARD_BALANCE
calculation_runs	Every calculation performed: name, date, input parameters (as JSON), results (as JSON), organization_id
support_tickets	Help desk tickets raised by users

PART 5 — User Interface: Step-by-Step Walk-through

5.1 Logging In

When the user navigates to the application, they see the Home page with the GannaApp branding. Clicking "Get Started" or "Log In" takes them to the Login page where they enter their registered email address and password. On successful login, a JWT (JSON Web Token) is issued by the server and stored in the browser's memory for the session.

The system has four user roles, each with different permissions:

Role	Permissions
superadmin	Full access to all organizations. Can create orgs, manage all users, view all tickets. Cannot see org calculation data.
admin	Full access to their own organization. Can manage team members, run calculations, upload data, view history.
user	Can run calculations, upload data, view results and history within their organization.
viewer	Read-only access. Can VIEW parameters and results but CANNOT run calculations or upload data.

5.2 Dashboard (Mission Control)

After logging in as a regular user (admin or user role), the Dashboard appears. This is the central hub of the application.

The Dashboard shows:

- A status bar showing how many of the 3 required files are loaded (e.g., "2/3 Files Loaded")
- A "Welcome Back" greeting with today's date
- A Consolidated Summary (if calculations exist) showing season-to-date performance
- Quick Action buttons: Go to Calculator, View History
- A Recent Calculations list showing the 5 most recent runs

The most important element is the "Manage Data" button (top right). Clicking it opens the Data Manager Modal, which is where all file uploads happen. Once all 3 required files are loaded, the button turns green and shows "Data Ready."

5.3 Data Manager Modal — Uploading Files

The Data Manager Modal shows a list of all uploadable data sources. Each has a status (LOADED in green or PENDING in gray) and action buttons:

Data Source	Upload Format	What Happens on Upload
Bonding File	CSV	Parsed and saved to backend. Defines center list and bonding quantities for the entire calculation.
Indent File	CSV	Parsed and saved. Historical indent records. Must cover from plant start date to at least T+2.
Purchase File	CSV	Parsed and saved. Historical purchase records. Must cover all purchases corresponding to the indents.
Yard Balance	CSV	Parsed and saved. When the current date is set, automatically

	(Date,Gate,Centre)	looks up the nearest date entry and fills in the Available Stock values.
Center Mapping	CSV (Source,Target)	NOT saved to backend. Applied immediately to merge source centers into target centers.

For Indent and Purchase files, there is also an "Append" button (+) that allows adding new daily records without overwriting the entire existing dataset. This is useful for updating files daily as the season progresses.

There is also an "Edit" button that opens a data grid showing all records, where the user can manually correct individual values.

5.4 Step 2 — Setting Parameters

The Calculator has 4 steps. Step 2 is where the user configures the calculation parameters:

Parameter	Default	Meaning
Current Date	Today	The date T for this calculation. All T+3, T-4, etc. references are relative to this date.
Plant Capacity	80%	The percentage of the daily target to actually plan for. At 80%, the effective requirement = daily target $\times$ 0.8.
Target Daily Run Rate	100,000 Qtl	The mill's maximum daily crushing capacity in Quintals.
Standard Stock – Gate	9,000 Qtl	Target yard balance for the Gate area (from standard yard balance file).
Standard Stock – Centre	6,000 Qtl	Target yard balance for all non-Gate centers combined.
Available Stock – Gate	Auto-filled	Actual current Gate yard balance — auto-populated from the uploaded Yard Balance file for the closest date $\leq$ current date. Can be manually overridden.
Available Stock – Centre	Auto-filled	Actual current Centre yard balance — same auto-fill logic.

The plant start date and season assumptions (total days, seasonal crushing capacity) are configured separately in the Settings modal (gear icon in the header).

5.5 Step 3 — Review and Calculate

The Review step shows a summary of all uploaded data and current parameters before the calculation runs. The user can see:

- How many records are in each uploaded file
- All current parameter values
- A confirmation that the data is ready

When the user clicks "Run Calculation," a dialog prompts for a name for this calculation run (e.g., "December 12 Morning Run"). This name appears in the history.

The calculation runs entirely in the browser in under 1 second for typical datasets (100+ centers, thousands of records). No data is sent to the server during calculation — only the results are saved to the server after completion.

5.6 Step 4 — Reading the Results

The results page has multiple tabs showing different views of the calculation:

Tab 1: Indent Recommendations (Main Table)

The primary output — a table with one row per center showing:

- Center name
- Bonding quantity
- Adjusted requirement (the target for this center after bonding share + stock adjustment)
- ForecastT3 (cane already coming from open indents)
- Indent to Raise (the recommended new indent quantity in Qtl)

Tab 2: Calculation Breakdown

A detailed step-by-step breakdown for each center showing every intermediate value: effective requirement, bonding percentage, requirement by bonding, stock adjustment, adjusted requirement, forecastT3, gap, overrun, target arrival, D1 weight, and final indent.

Tab 3: Maturity Analysis (Recent)

Shows the D-weight computation details for the last 4 closed indents (T-7 to T-4) per center. The user can see exactly which indents were used, the raw D1-D4 rates per indent, and the simple averages.

Tab 4: Full Season Maturity

A season-wide maturity analysis showing D-weights computed from ALL indents since the plant start date. Used as the fallback when recent data is insufficient.

Tab 5: Open Indent Matrix

A calendar-style matrix showing, for each center and each day in a ~10-day window (T-3 to T+6), the expected or actual cane arrivals. Actual figures (past dates) come from the purchase file. Forecast figures (future dates) are computed using the D-weights applied to the relevant open indents. The new TBD (To-Be-Determined) indent for T+3 is also included in the forecast.

Tab 6: Forecast Breakdown

A detailed view of how forecastT3 was computed for each center — showing each contributing indent (T+0, T+1, T+2), its quantity, the applicable D-weight, and the resulting contribution in Quintals.

Simulation Mode:

From the results page, the user can run WHAT-IF scenarios. They can change a parameter (e.g., "what if plant capacity drops to 60%?") and run a scenario calculation. Scenarios are saved separately in history with an "S" badge so they are clearly distinguishable from real calculation runs.

5.7 History Page

All completed calculations (real and scenario) are saved and appear on the History page. The user can:

- Browse all past calculations with their name, date, and timestamp
- Click any past calculation to view its full results (as of when it was run)
- Delete individual calculations

History is organization-scoped — one organization's users cannot see another organization's history.

PART 6 — Technical Deep Dive: Code Walk-through

6.1 calculationService.ts — The Brain

All calculation logic lives in the single file `frontend/services/calculationService.ts`. The main exported function is `calculateRecommendedIndents(inputs)`, which accepts a `CalculationInputs` object and returns a `CalculationResults` object.

The function flow:

1. `normalizeBonding()` — Apply center mapping to bonding data. Aggregate bonding for merged centers. Detect if a center is "Gate" by checking if its name contains the word "GATE".
2. `normalizeIndents()` — Apply center mapping to indents. Deduplicate same center+date combinations by summing quantities.
3. `normalizePurchases()` — Apply center mapping to purchases.
4. `deriveMaturityWeights()` — For each center in the bonding data, find recent closed indents (T-7 to T-4), compute per-indent D-rates, take simple average. Fall back to season average if recent data is sparse.
5. `calculateForecasts()` — For each center, find open indents (T+0, T+1, T+2) and apply D-weights to forecast T+3 arrivals.
6. Compute season-wide overrun from total purchases / total indents.
7. For each center: compute adjusted requirement, subtract forecast T3, divide by (1+overrun) then by `D1_avg` to get the final indent.
8. `calculateOpenIndentMatrix()` — Build the full calendar matrix for the results visualization.
9. Return everything as a `CalculationResults` object.

6.2 Data Normalization — Handling Center Mapping

Center mapping is applied FIRST, before any calculation. The `normalizeIndents` function:

10. Reads each row of the raw indent CSV.
11. Looks up the center's Code in the `centerMapping` dictionary. If a mapping exists (e.g., "28"→"23"), replaces the `centreId` with the target.
12. Groups by (`centreId`, `indent_date`) and sums quantities — so if both original center 23 and mapped center 28 had indents on the same date, they are summed.
13. Returns a clean array of {`centreId`, `raisedFor`, `qty`} objects.

The same mapping logic is applied to `normalizePurchases` and `normalizeBonding`. The result is that downstream calculations never "see" the original source center IDs.

6.3 D-Weight Computation in Code

The `analyzeIndentMaturity` function:

Given a single indent and its associated purchases, this function buckets each purchase into D1, D2, D3, or D4 based on (`purchaseDate` – `indentDate`) in days:

- $\text{dayDiff} \leq 0 \rightarrow \text{D1}$ (arrived on or before indent date)
- $\text{dayDiff} = 1 \rightarrow \text{D2}$ (arrived exactly 1 day after indent date)
- $\text{dayDiff} = 2 \rightarrow \text{D3}$ (arrived exactly 2 days after indent date)

- $\text{dayDiff} \geq 3 \rightarrow \text{D4}$ (arrived 3 or more days after indent date)

All D-weight dates are computed in UTC midnight to avoid timezone edge cases that could cause off-by-one day errors.

The `deriveMaturityWeights` function:

This function builds a `Map<centreId, DWeights>` for all centers. For each center:

14. Find all closed indents ($\text{date} \leq T-4$) from the recent window ($\text{date} \geq T-7$).
15. For each such indent, call `analyzeIndentMaturity` to get D1-D4 purchase sums.
16. Compute per-indent D-rates: $\text{D1_rate} = \text{D1_purchases} / \text{indent_qty}$.
17. Average the D-rates across all recent indents using simple average.
18. If recent average $\text{D1} < 0.1$ (a fallback threshold indicating insufficient data), use the season-wide average instead.
19. Store the final $\{\text{d1}, \text{d2}, \text{d3}, \text{d4}\}$ weights for this center.

6.4 Forecast Computation in Code

The `calculateForecasts` function:

For each center in the bonding array:

20. Look up the indent for $T+2$ ($\text{date} = \text{today} + 2 \text{ days}$). This is the indent whose D2 cane arrives on $T+3$.
21. Look up the indent for $T+1$. D3 cane arrives on $T+3$.
22. Look up the indent for $T+0$ (today). D4 cane arrives on $T+3$.
23. Compute: $\text{centerForecast} = \text{indent}(T+2) \times \text{D2} + \text{indent}(T+1) \times \text{D3} + \text{indent}(T+0) \times \text{D4}$.
24. Sum all center forecasts for a `globalForecastT3`.

If any of the $T+0$, $T+1$, $T+2$ indents is missing from the upload (e.g., the mill did not indent that center on that day), the corresponding contribution is 0. This will increase the gap and thus the recommended new indent.

6.5 Final Indent Computation in Code

The main loop (`calculateRecommendedIndents` function):

After D-weights and forecasts are computed, for each center:

25. Compute $\text{bondingPercentage} = \text{center.qty} / \text{totalBonding}$.
26. Compute $\text{requirementByBonding} = \text{effectiveRequirement} \times \text{bondingPercentage}$.
27. Determine stock type (Gate vs non-Gate) and compute `stockAdjustment`.
28. $\text{adjusted_requirement} = \text{requirementByBonding} + \text{stockAdjustment}$.
29. Look up this center's `forecastT3` from the forecast results.
30. $\text{netRequirement} = \max(0, \text{adjusted_requirement} - \text{forecastT3})$.
31. $\text{overrunPercentage} = (\text{totalPurchaseQty} / \text{totalIndentQty}) - 1$ [season-wide].
32. $\text{targetArrival} = \text{netRequirement} / (1 + \text{overrunPercentage})$.
33. $\text{d1Weight} = \text{centerWeights.get(centreId).d1}$ [from step 3.2].
34. $\text{finalIndent} = \text{d1Weight} > 0 ? \text{targetArrival} / \text{d1Weight} : 0$.

The finalIndent is stored in the IndentResultRow for display in the results table. The detailed intermediate values are stored in IndentCalculationBreakdownRow for the Calculation Breakdown tab.

PART 7 — Testing and Validation

7.1 The Excel Reference Model

The ground-truth algorithm comes from the file `IndentingModel_TestFile.xlsxsm` — a Microsoft Excel macro-enabled workbook that the mill previously used manually for indent planning. This Excel file contains the exact same mathematical formulas that GannaApp replicates.

The Excel file has the following key sheets:

Sheet Name	Contents
Forecast_Day1	The core calculation for the GATE center. Shows the Closed Indent Matrix (last 4 closed indents with D1-D4 rates), Open Indent Matrix (forecast from existing open indents), and the final TBD indent recommendation.
Maturity Report	Season-wide D-weights per center (used as fallback when recent data is thin). Also contains the center maturity mapping table.
Waterfall	A large matrix showing actual purchase quantities for every center × every day — the raw data behind the maturity analysis.
Report	Input parameters (standard stock, available stock, plant capacity, daily requirement) and the final per-center indent recommendations.
Summary	Season-to-date performance metrics.
Sheet1	Clean final output table of recommended indents per center.

The application's algorithm has been verified by manually tracing through the Excel formulas for the GATE center on January 27, 2023, and confirming the numbers match exactly: `finalIndent` = 58,673 Qtl in both the Excel and the application's formula.

7.2 Test Case — December 12, 2025 (GATE)

The following table shows the expected calculation values for the GATE center when the calculation is run for `T` = December 12, 2025, using the "11Dec" test data files (`11Dec_Season Indent.csv` and `11Dec_Purchase.csv`):

Calculation Step	Value	How It Was Computed
D-Weight Data Source	Dec 5, 6, 7, 8 indents	T-7 to T-4 = Dec 5 to Dec 8
D1 average	0.4460	Simple avg of (D1/indent) across 4 indents
D2 average	0.3877	Simple avg of (D2/indent) across 4 indents
D3 average	0.2123	Simple avg of (D3/indent) across 4 indents
D4 average	0.0436	Simple avg of (D4/indent) across 4 indents
Season overrun	12.44%	(4,631,765 total purchases) ÷ (4,119,192 total indents) – 1
Effective requirement	80,000 Qtl	100,000 × 80% plant capacity
GATE bonding share	23.90%	4,126,404 ÷ 17,266,808 total bonding
Requirement by bonding	19,118 Qtl	80,000 × 23.90%
Standard stock (Gate)	9,000 Qtl	From standard yard balance file
Available stock (Gate)	1,767 Qtl	Dec 11 value from <code>Yard Balance_main.xlsx</code>
Stock difference	+7,233 Qtl	9,000 – 1,767 (below target → increase indent)
Stock adjustment	+7,233 Qtl	Full gate deficit goes to GATE (only Gate center)
Adjusted requirement	26,351 Qtl	19,118 + 7,233
T+0 indent (Dec 12)	23,022 Qtl	GATE indent on Dec 12

T+1 indent (Dec 13)	27,990 Qtl	GATE indent on Dec 13
T+2 indent (Dec 14)	29,610 Qtl	GATE indent on Dec 14
ForecastT3	18,424 Qtl	$29,610 \times 0.3877 + 27,990 \times 0.2123 + 23,022 \times 0.0436$
Gap	7,928 Qtl	$26,351 - 18,424$
Target arrival	7,050 Qtl	$7,928 \div 1.1244$
Final GATE indent	15,808 Qtl	$7,050 \div 0.4460$

Note: The actual output will depend on the exact parameter values entered by the user (daily requirement and plant capacity). The above uses the typical defaults. The user should set these to match what they are using in their Excel model for a fair comparison.

7.3 How to Run the Test

To test GannaApp against the Excel model for December 12:

35. Export the Yard Balance_main.xlsx file as a CSV: File → Save As → CSV. The columns should be: Date, Gate, Centre. Dates should be in DD-MM-YYYY format.
36. Log in to GannaApp. Open the Data Manager Modal.
37. Upload bonding_data.csv as "Bonding File."
38. Upload 11Dec_Season Indent.csv as "Indent File."
39. Upload 11Dec_Purchase.csv as "Purchase File."
40. Upload the yard balance CSV as "Yard Balance."
41. Upload Center Mapping.csv as "Center Mapping."
42. Close the Data Manager. Navigate to the Calculator (Step 2 — Parameters).
43. Set Current Date = 12-12-2025.
44. Verify: Standard Stock Gate = 9,000; Standard Stock Centre = 6,000.
45. Verify: Available Stock Gate and Centre have been auto-filled from the yard balance file (Dec 11 values).
46. Set Plant Capacity and Target Daily Run Rate to match your Excel model inputs.
47. Click "Next: Review" then run the calculation.
48. Compare the GATE row's "Indent to Raise" value with your Excel model output for December 12.

Any significant discrepancy (more than a few Quintals) would indicate a data difference between the files loaded in the app vs. those used in the Excel, OR a difference in parameter values (daily requirement, plant capacity).

PART 8 — Roles, Permissions, and Multi-Tenancy

GannaApp is built as a multi-tenant SaaS (Software-as-a-Service) application. Multiple sugarcane mills can use the same application instance, each completely isolated from the others in terms of data and users.

8.1 Organization Isolation

Every user belongs to exactly one organization (mill). When they log in:

- They can only see their organization's uploaded data files
- They can only see their organization's calculation history
- They can only see and manage their organization's team members
- Data from other organizations is completely inaccessible to them

At the database level, all data tables include an `org_id` column. Every query filters by the authenticated user's organization ID, enforced on the server side.

8.2 Role Details

Role	Who Uses It	Key Capabilities
superadmin	System administrator (Anthropic/developer)	Creates and manages organizations. Creates admin users. Sees all support tickets. Cannot see any organization's operational data.
admin	Mill manager or IT administrator	Full control within their org. Can create/delete team members. Can upload all data files. Can run calculations. Can manage the center mapping and season settings.
user	Indent planner, operations staff	Can upload files, run calculations, view all results and history. Cannot manage team members.
viewer	Read-only stakeholder (senior management, auditor)	Can view all results and history. Cannot upload data, run calculations, or change any settings. All inputs are disabled (grayed out).

8.3 Authentication Flow

Authentication uses stateless JWT tokens:

49. User submits email + password to POST `/api/auth/login`.
50. Server verifies password against bcrypt hash stored in database.
51. Server issues a JWT containing: user ID, email, role, and organization ID.
52. JWT is stored in React state (in memory — NOT in localStorage for security).
53. Every subsequent API call includes the JWT in the Authorization header: "Bearer <token>."
54. The `requireAuth` middleware on the server verifies and decodes the JWT for every protected route.
55. When the user logs out or refreshes the page, the token is lost and they must log in again.

PART 9 — Frequently Asked Questions

Q: Why does the app need purchase data if I only want to compute future indents?

A: Purchase data is essential for computing the D-weights (maturity rates). Without knowing how cane from past indents actually arrived over time, the algorithm cannot predict how much of today's indent will arrive on T+3 vs. T+4, T+5, etc. The D-weights are derived ENTIRELY from historical purchase-vs-indent comparisons.

Q: What happens if a center has no recent indent data (e.g., it just joined mid-season)?

A: The algorithm falls back to the SEASON-WIDE D-weight averages for that center. If even season-wide data is missing (new center with no history at all), D-weights will be 0 and the final indent recommendation will be 0. The user should verify such centers manually.

Q: Why is the overrun computed from the WHOLE season, not just recent indents?

A: The Excel reference model uses season-wide overrun, and the application matches this. Season-wide overrun is more statistically stable than recent overrun because it averages out day-to-day fluctuations. Using only 4 recent indents could be noisy if there was unusual weather or road conditions in the past week.

Q: The "Available Stock – Gate" field says auto-filled. What if the yard balance file does not have today's date?

A: The auto-fill logic finds the MOST RECENT entry with a date on or before the current calculation date. So if the file only goes up to December 11 and the calculation date is December 12, it uses December 11's values. This is acceptable because the yard is measured at end-of-day, and yesterday's end-of-day balance is the best proxy for today's available stock.

Q: What is the difference between "Indent Date" and the date the indent was placed?

A: The "Indent Date" in this application is the TARGET DELIVERY DATE — the date on which the indent's D1 cane should start arriving at the mill. It is 3 days in the future from when the indent is placed. When a planner runs the calculation on December 12, the resulting indent is "for" December 15 (the Indent Date = T+3).

Q: Can the app handle mills with very few centers (e.g., 5-10 centers)?

A: Yes. The algorithm works for any number of centers. With fewer centers, the results are faster to compute and easier to verify manually.

Q: The D1 weight for some centers is extremely low (< 5%). Is this a problem?

A: A very low D1 weight means almost no cane from that center arrives on the indent date itself — it all comes 1-4 days later. This is normal for distant centers where transport takes longer. The algorithm handles this correctly. The concern arises only if $D1 = 0$ for ALL recent indents, which triggers the fallback to season averages. If season averages are also 0 (no historical data), the center gets a 0 indent recommendation.

Q: What is "Overrun" in plain business terms?

A: Overrun means you received more cane than you ordered. If you placed an indent for 100 Qtl and 112 Qtl arrived, the overrun is 12%. This happens because farmers sometimes deliver more than their agreed quantity, or cross-deliveries occur. The algorithm uses overrun to calibrate: since you know you'll receive slightly more than you order, you should order slightly less than the exact gap.

Q: What does "Bonding" mean in the context of the algorithm?

A: Bonding determines each center's PROPORTIONAL SHARE of the total daily requirement. It is not used to compute D-weights (which come only from actual indent/purchase history). It only answers the question: "Of the 80,000 Qtl I need today, how much should center X be responsible for?" The answer is: $X's\ bonding \div total\ bonding \times 80,000$.

Q: What happens if I forget to update the yard balance file daily?

A: The application uses the most recent entry in the yard balance file. If the file was last updated 3 days ago, the stock adjustment will be based on 3-day-old data. This may slightly distort the indent. It is best practice to update the yard balance CSV daily before running the calculation.

Q: Can multiple users in the same organization run the calculation simultaneously?

A: Yes. Each calculation is independent and runs in the browser. The results are saved separately to the database with individual IDs. The last run saved will appear at the top of history. There is no conflict resolution needed because calculations are read-only operations on shared data.

Q: How does the Scenario / What-If feature work?

A: From the Results page, the user can click "Run Scenario." They can modify any parameter (e.g., reduce plant capacity to 60%, or change the available stock) and rerun the calculation. The scenario result is saved in history with an "S" badge, clearly distinguishing it from a real production run. This allows planners to explore the impact of events like mill maintenance or heavy rain without affecting the main calculation.

APPENDIX A — Complete Formula Reference

All formulas in compact notation. Variables:

T	= current date
N	= number of centers
i	= center index
B_i	= bonding quantity for center i
B_{total}	= sum of all bonding
B_{gate}	= total bonding of Gate-type centers
B_{centre}	= total bonding of non-Gate centers
DR	= target daily run rate (e.g., 100,000 Qtl)
PC	= plant capacity as percentage (e.g., 80)
SS_{gate}	= standard stock (Gate)
SS_{ctr}	= standard stock (Centre)
AS_{gate}	= available stock (Gate) from yard balance
AS_{ctr}	= available stock (Centre) from yard balance
OR	= season-wide overrun = $(\Sigma Purchases / \Sigma Indents) - 1$
$D1_i \dots D4_i$	= D-weights for center i
$I(i,t)$	= indent quantity for center i on date t

Formula Name	Expression
Effective Requirement	$EffReq = DR \times (PC \div 100)$
Requirement by Bonding (center i)	$ReqBond_i = EffReq \times (B_i \div B_{total})$
Stock Diff (Gate)	$StockDiff_{gate} = SS_{gate} - AS_{gate}$
Stock Diff (Centre)	$StockDiff_{ctr} = SS_{ctr} - AS_{ctr}$
Stock Adjustment (Gate center i)	$StockAdj_i = StockDiff_{gate} \times (B_i \div B_{gate})$
Stock Adjustment (non-Gate center i)	$StockAdj_i = StockDiff_{ctr} \times (B_i \div B_{centre})$
Adjusted Requirement	$AdjReq_i = ReqBond_i + StockAdj_i$
D-Weight (per indent j)	$D1_{ij} = D1_{purchases_{ij}} \div IndentQty_{ij}$ (similarly D2,D3,D4)
Average D-Weight (center i)	$D1_i = SimpleAvg(D1_{ij})$ over last 4 closed indents
Forecast T+3 (center i)	$ForecastT3_i = I(i,T+2) \times D2_i + I(i,T+1) \times D3_i + I(i,T) \times D4_i$
Gap	$Gap_i = \max(0, AdjReq_i - ForecastT3_i)$
Season Overrun	$OR = (\Sigma \text{ all purchases}) \div (\Sigma \text{ all indents}) - 1$
Target Arrival	$TargetArrival_i = Gap_i \div (1 + OR)$
Final Indent	$FinalIndent_i = TargetArrival_i \div D1_i$ (= 0 if $D1_i = 0$)

APPENDIX B — Glossary of Technical Terms

Term	Definition
API	Application Programming Interface — a set of HTTP endpoints the frontend uses to communicate with the backend server
CSV	Comma-Separated Values — a plain text file format where each row is a record and columns are separated by commas
D-Weight	Maturity fraction — the proportion of an indent that arrives on a specific day (D1, D2, D3, or D4)
JWT	JSON Web Token — an encrypted token that proves the user's identity without the server needing to store session data
Overrun	The excess of actual purchases over indented quantities, expressed as a fraction: (purchases/indents) - 1
Plant Capacity	The percentage of the mill's maximum crushing capacity that is actually targeted. At 80%, the mill plans to crush 80% of its maximum.
Quintal (Qtl)	100 kilograms — the standard unit of measurement for sugarcane in India
React	A JavaScript library for building user interfaces from reusable components
REST API	A style of API design using standard HTTP methods (GET, POST, DELETE) to perform operations on resources
SaaS	Software-as-a-Service — a software delivery model where the application is hosted centrally and accessed via a web browser
Simple Average	The arithmetic mean: sum all values and divide by the count. No value is weighted more than any other.
T+3	Three days from the current date T — the target delivery date for the indent raised today
TypeScript	A strongly-typed superset of JavaScript that catches programming errors before the code runs
Vite	A modern frontend build tool that makes development faster with hot-reload and optimized production builds
Weighted Average	An average where larger values carry more influence — NOT used in this application's D-weight calculation
Yard Balance	The physical stock of sugarcane sitting in the mill's yard at any given time, measured in Quintals